

Stitch and Blending

Dr. Praisan Padungweang, Collage of Computing, KKU

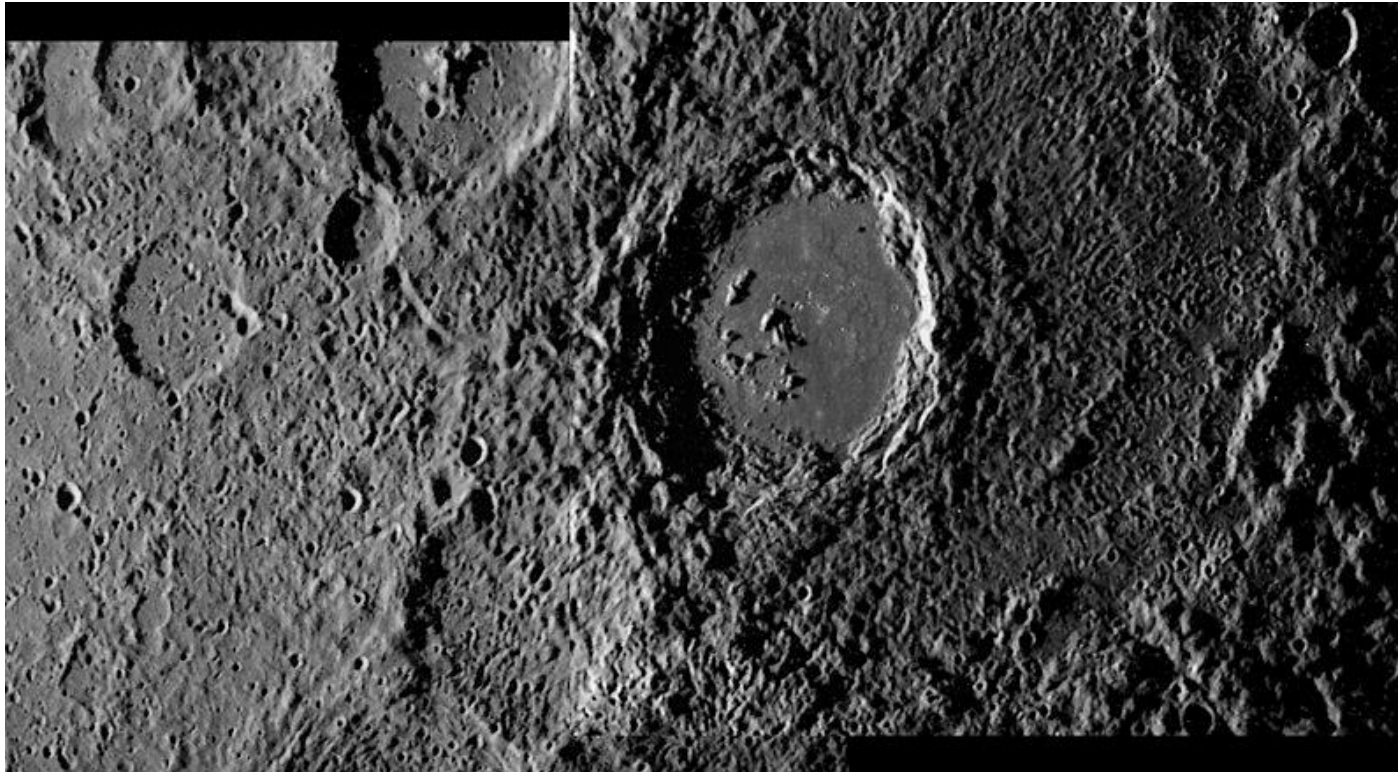
Credit: Steve Elston



Image Stitching

Image stitching has a long history

- Performed manually from 1920s into 1980s
- Example, a mosaic image of the Hitomaro Mars crater – Mariner 9, 1970

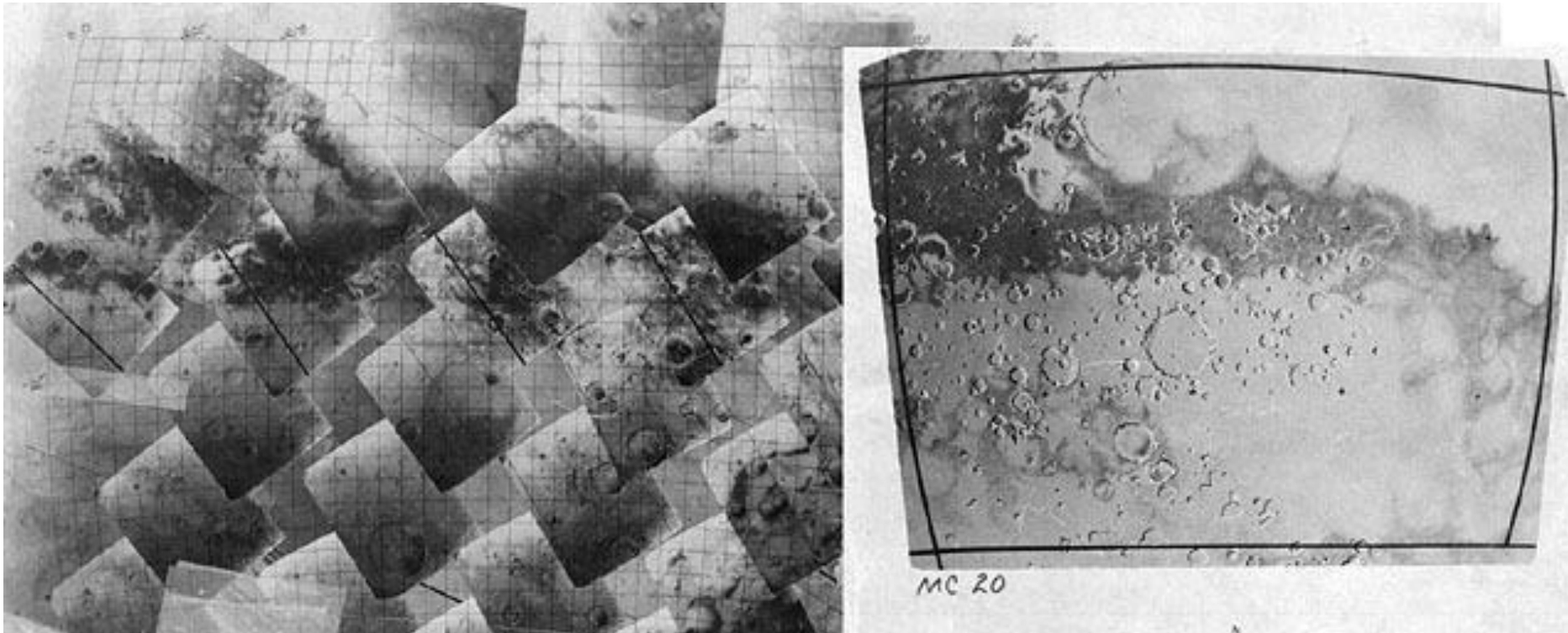


Credit, Wikipedia
commons

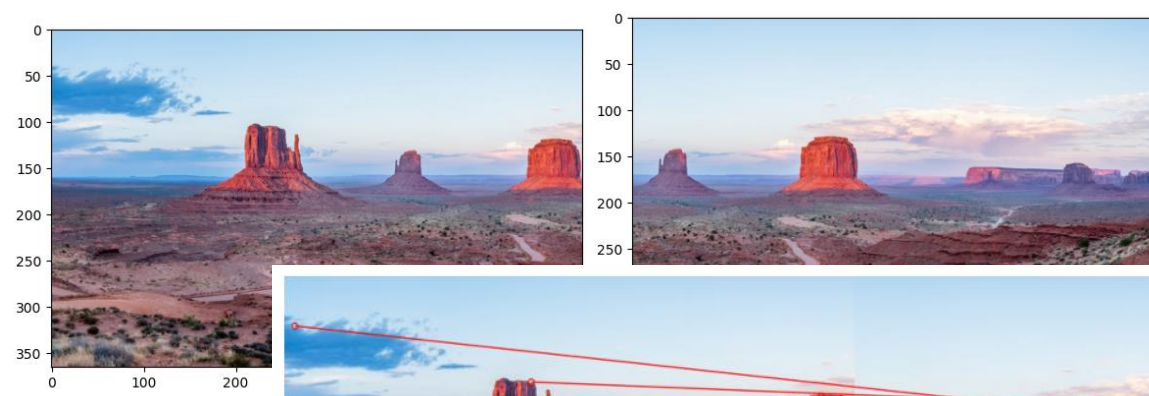
Image Stitching

Image stitching has a long history

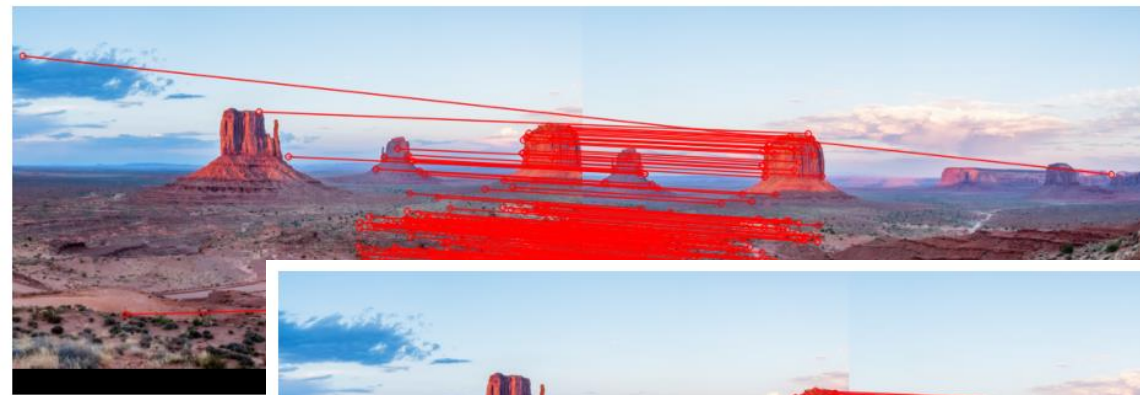
- Before CV methods photo mosaics were manually assembled for decades
- Example, assembling Mariner 9 Mars images in 1970



Credit, The
Planetary Society



1. Matching



2. Outlier removal



3. Mapping



4. Blending





Review of Interest Points and Descriptors

Interest points and descriptors

Need interest points and descriptor which are scale and rotationally invariant

- Interest points (key points)
 - Corners, edges, line intersections, etc.
 - High gradients
 - Ideally scale and rotationally invariant
- Descriptors
 - Uniquely identify interest points
 - Use image characteristics within patch around interest point

Interest points and descriptors

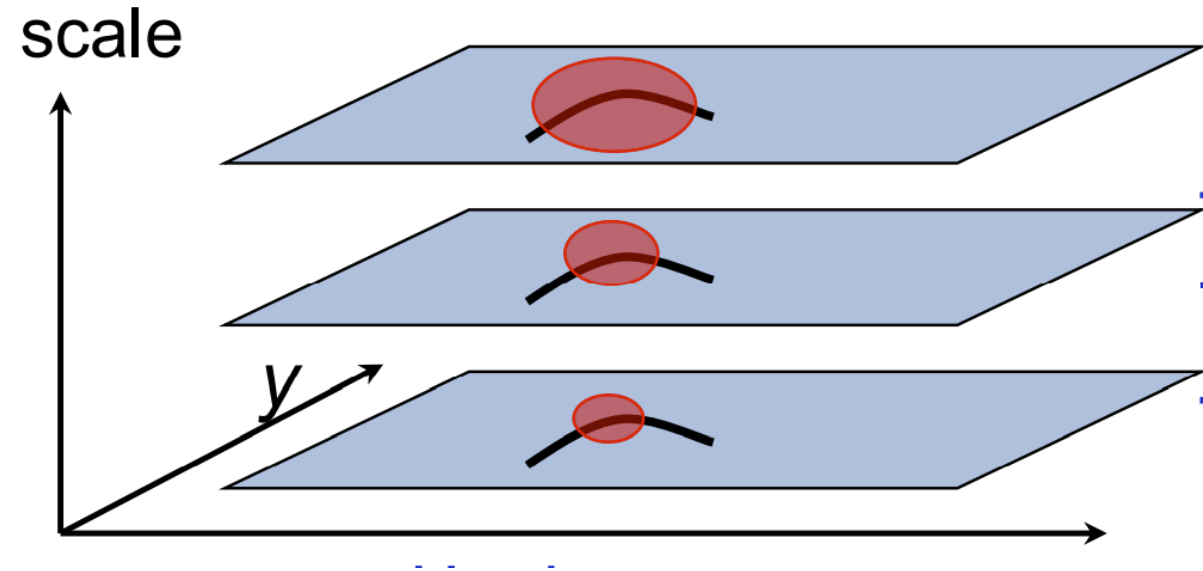
Review of interest points and descriptors

- Rotationally invariant interest point detectors
 - Harris, etc.
 - Increasingly neural networks; [example with Keras](#)
- Descriptors
 - SHIFT
 - FAST
 - BRIEF
 - etc

Multi-scale interest point detection

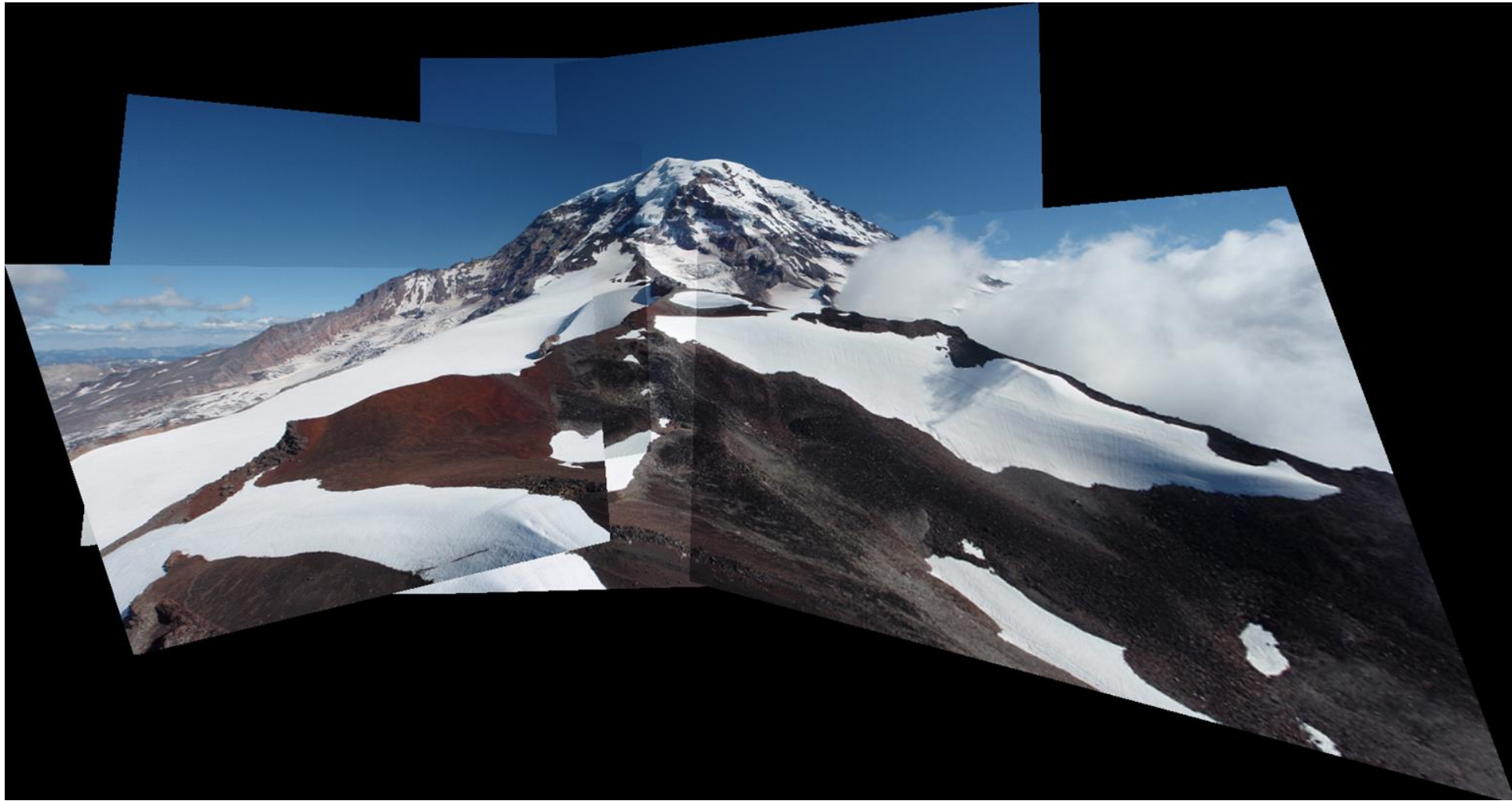
Pyramid structure enables extraction of interest points at several scales

- Pyramid scales from fine to coarse in octaves
- Features at each scale extracted
- Can use a variety of interest point descriptor algorithms
 - SIFT
 - Neural network



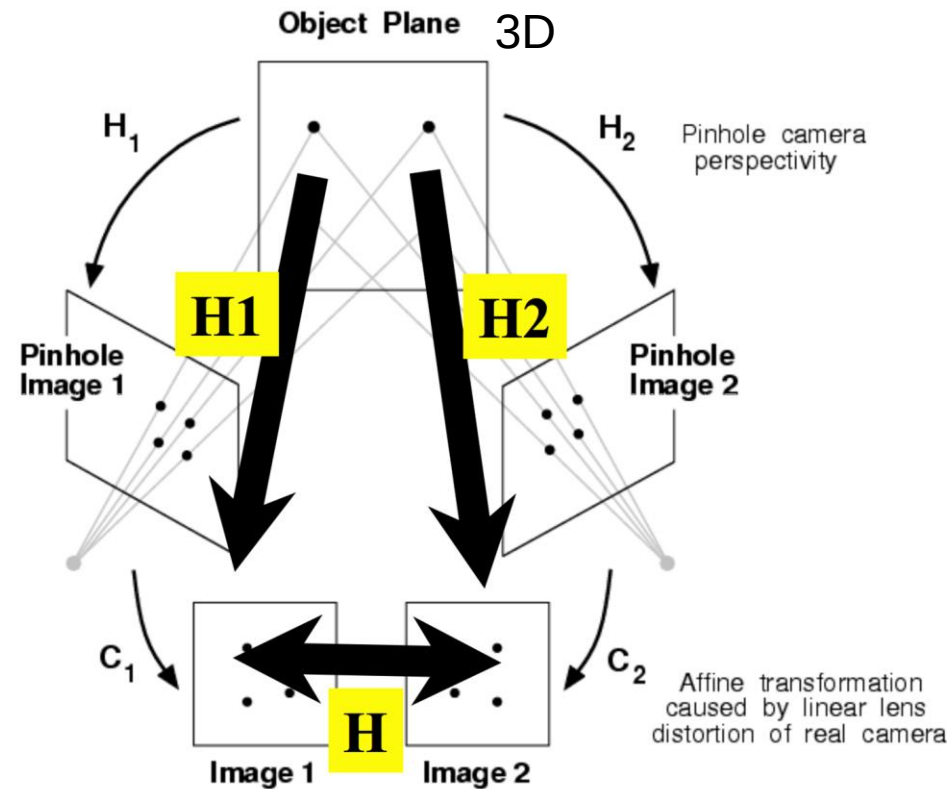


Planar Transformations for Image Stitching



Stitching panoramas:

- We know homography is right choice under certain assumption:
 - Assume we are taking multiple images of planar object



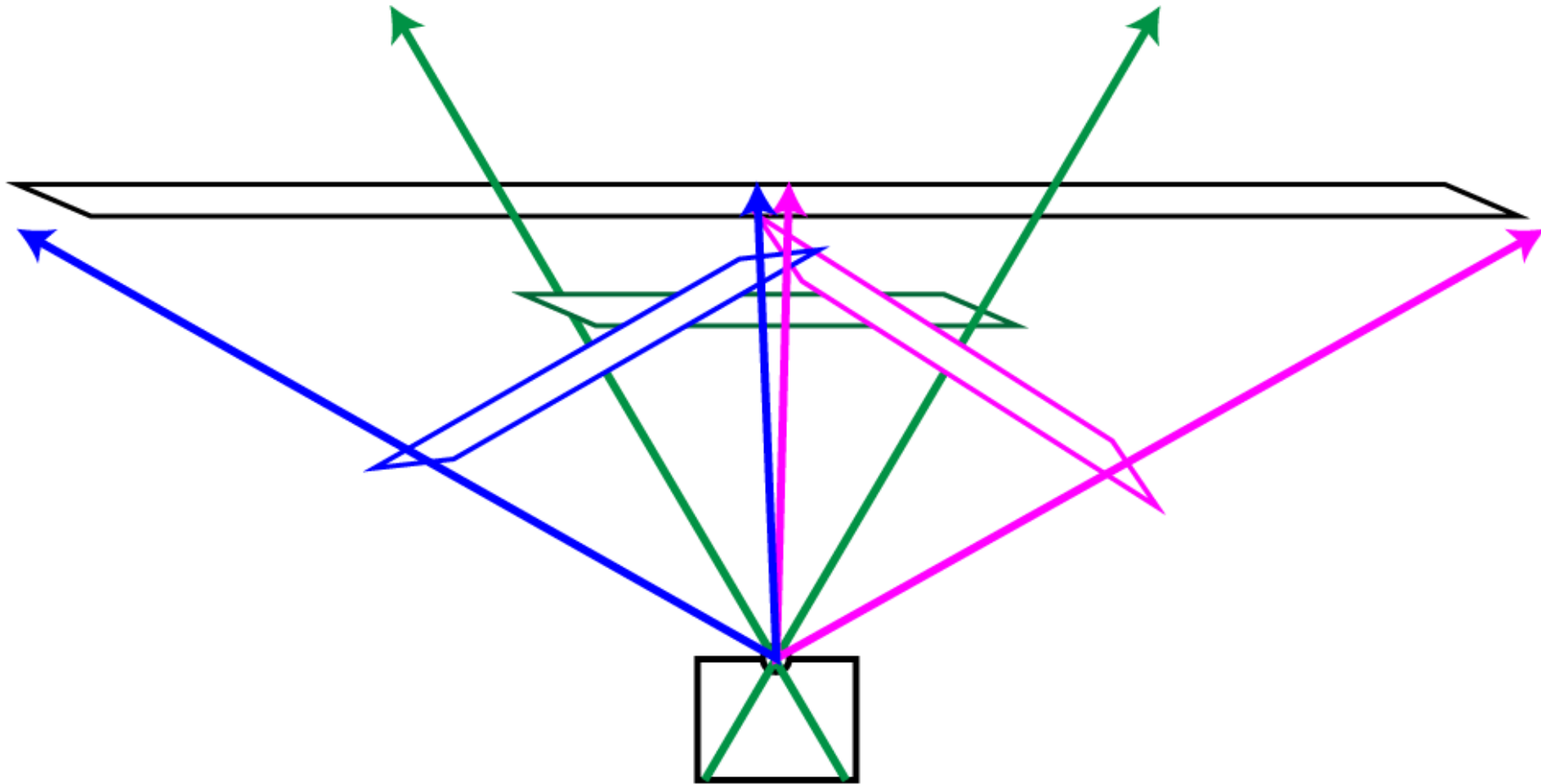
homography H

Planar transformations

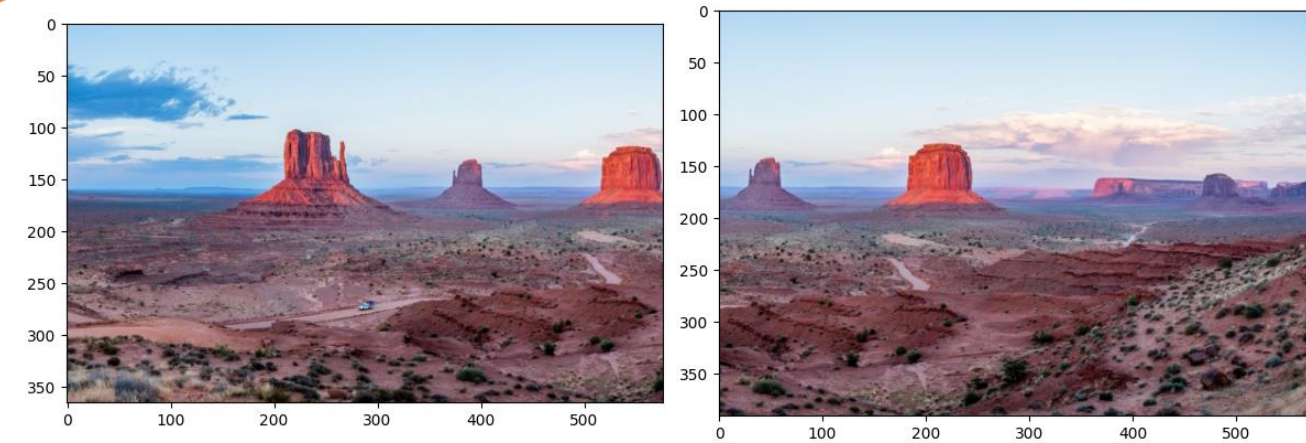
Can use different approximations for image stitching

- Cylindrical coordinates
 - Assume camera rotates about an axis
- Spherical coordinates
 - Allows camera to rotate unrestricted
- Planar
 - Good approximation if objects far from camera
 - Good approximation for small changes in camera pose
 - Our focus here for simplicity

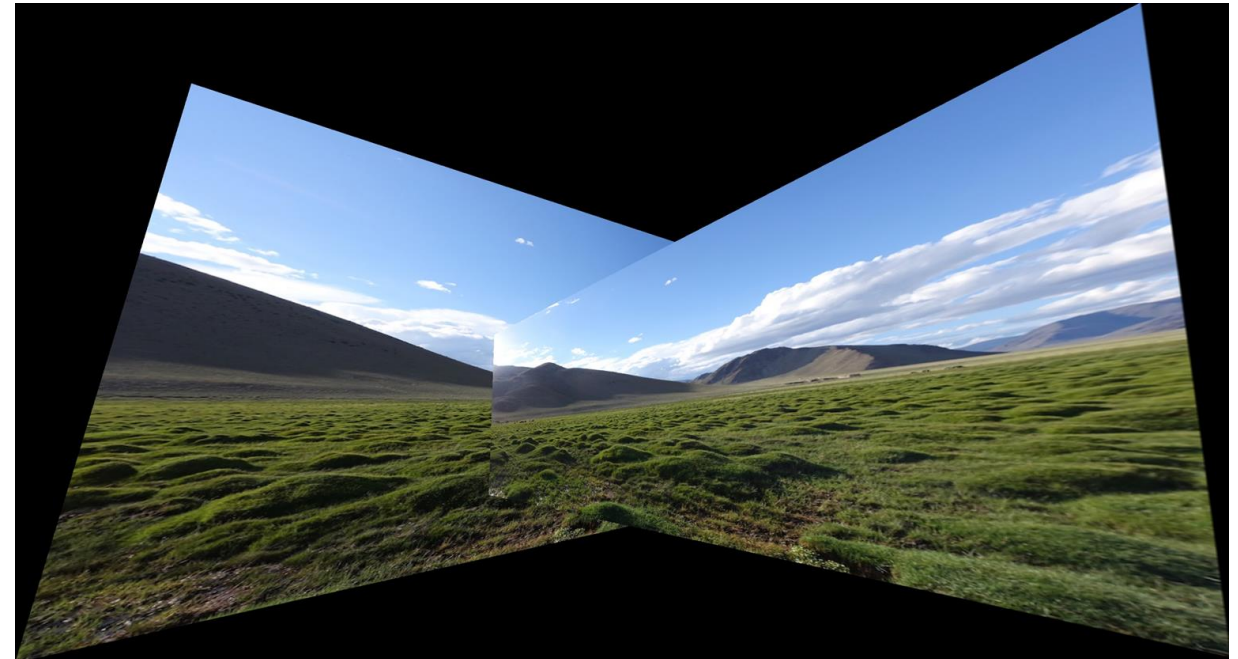
Mapping to flat plane



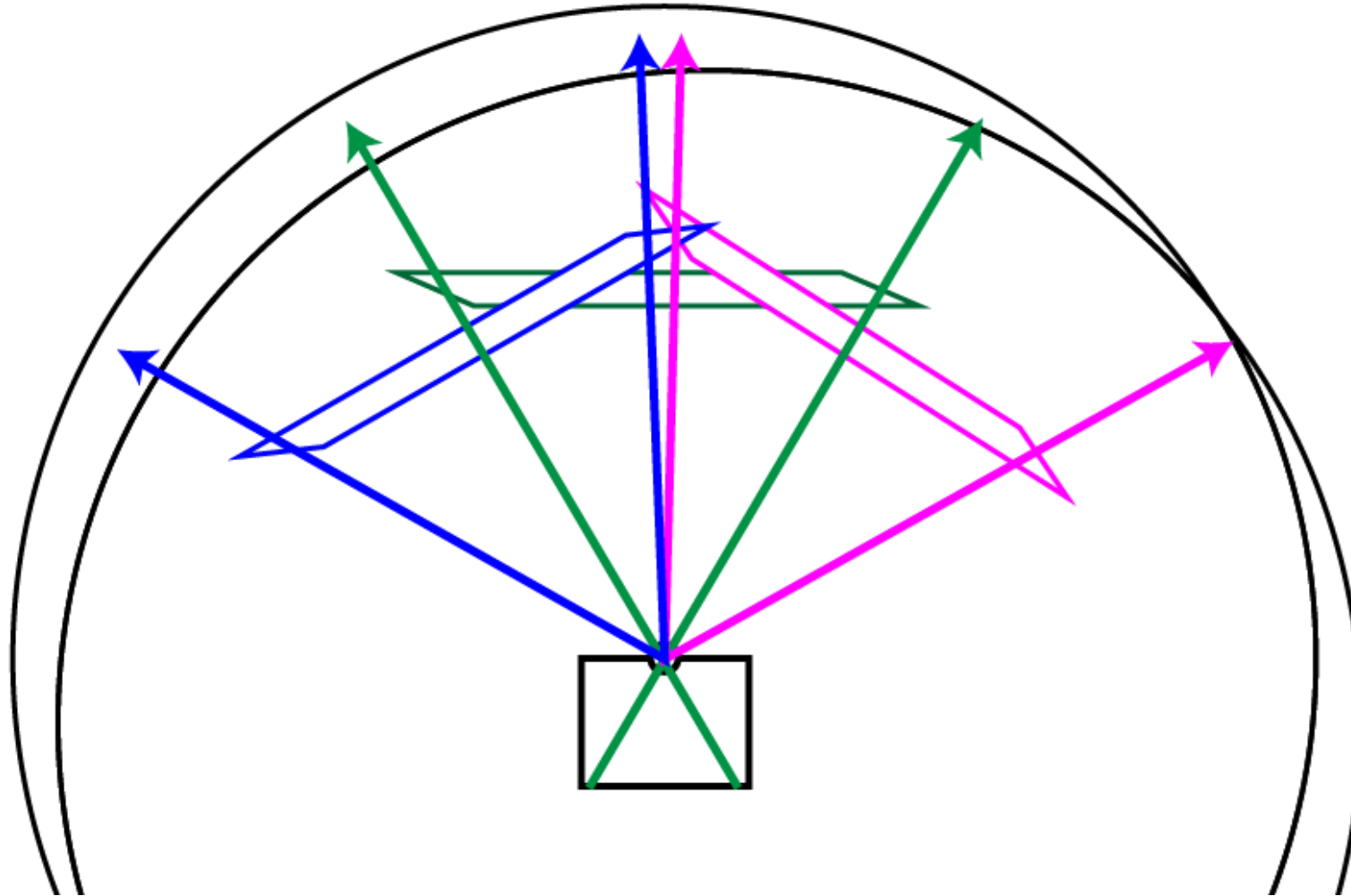
bad for big panoramas Fails :-)



bad for big panoramas ->>



Cylinders mapping



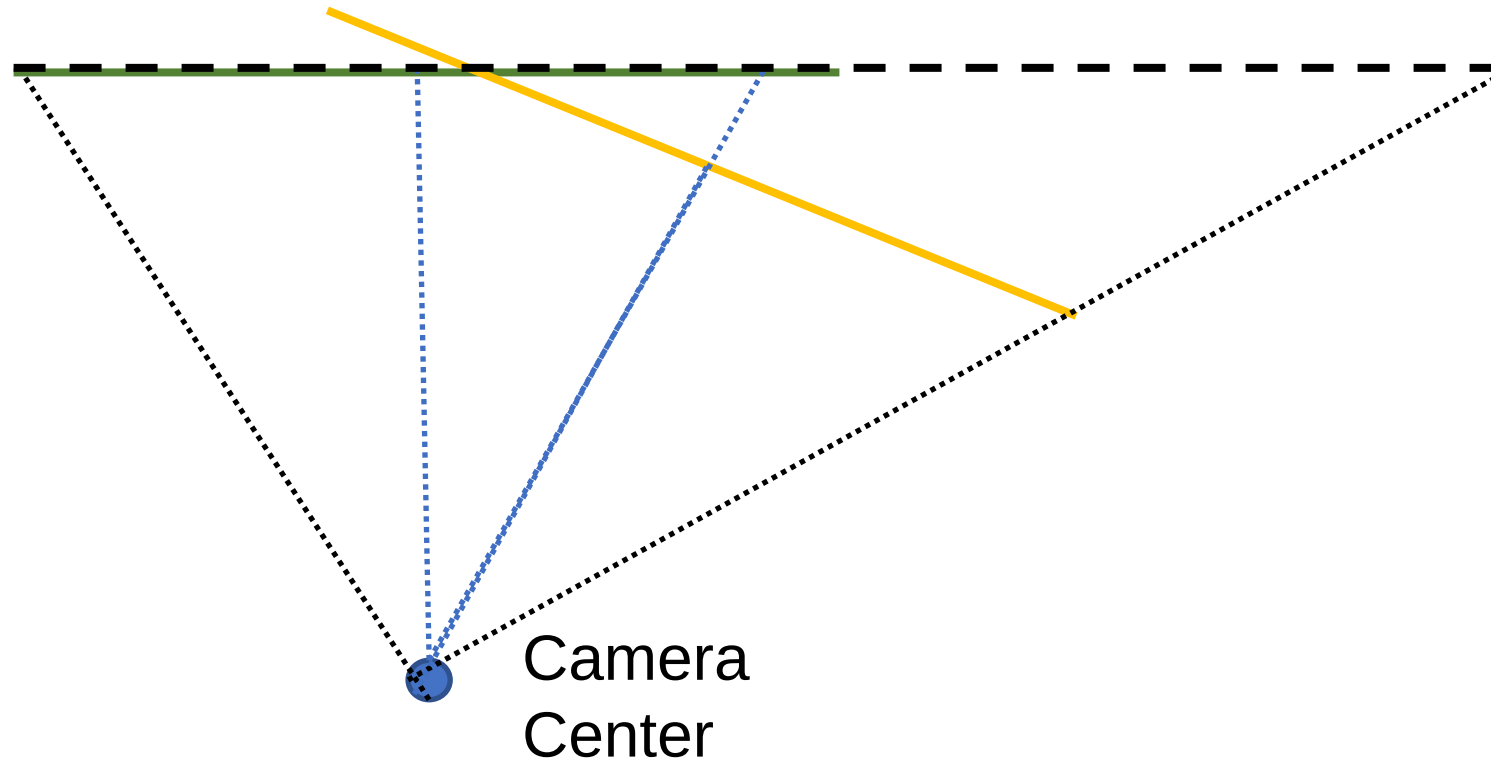
Cylinders mapping



Planar transformations

Planar transformation maps one image plane to another

- Two images planes from a camera centre
- Project one image plane onto the other



Four Transform Problems

Single planar homography imposes limitations

- Model is restricted to planar transformations
- Model assumes:
 - Low **parallax**
 - All objects in image are at a **constant distance from image plane**
 - Only **small changes in camera pose**
- More complex model can accommodate more general transformations
 - Use non-planar geometry
 - Apply projective transform
 - Beyond the scope of this course



Solving for Homographies

Review of Transformations

$f'(x', y')$

	0	1	2	3	4
0	1	1	1	1	1
1	1	1	0	0	0
2	1	1	1	0	1
3	1	1	1	0	1
4	1	1	1	0	1



$f(x, y)$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
3	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
4	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1
5	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1
6	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1
7	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1
8	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1
9	1	1	1	1	1	1	1	1	1	0	0	0	1	1	1



$f(x, y)$

	0	1	2	3	4
0	0	0	0	1	1
1	1	0	1	1	1
2	1	0	1	1	1
3	1	0	1	1	1
4	1	1	1	1	1

$$f(x, y) = f'(x + 2, y + 1)$$

$$\begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & \tau_x \\ 0 & 1 & \tau_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$f(x, y) = f'\left(\frac{x}{\frac{14}{4}}, \frac{y}{\frac{9}{4}}\right)$$

$$f(6, 2) = f'(1.71, 0.89) \cong f'(2, 1)$$

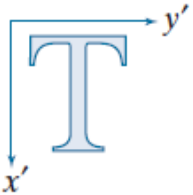
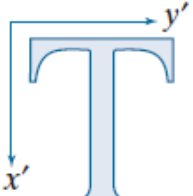
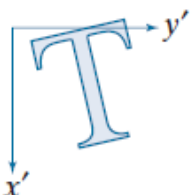
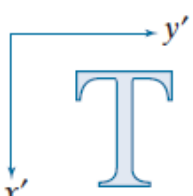
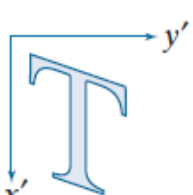
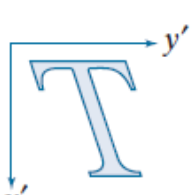
$$\begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{14}{4} & 0 & 0 \\ 0 & \frac{9}{4} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 2 \\ 1 \end{bmatrix}$$

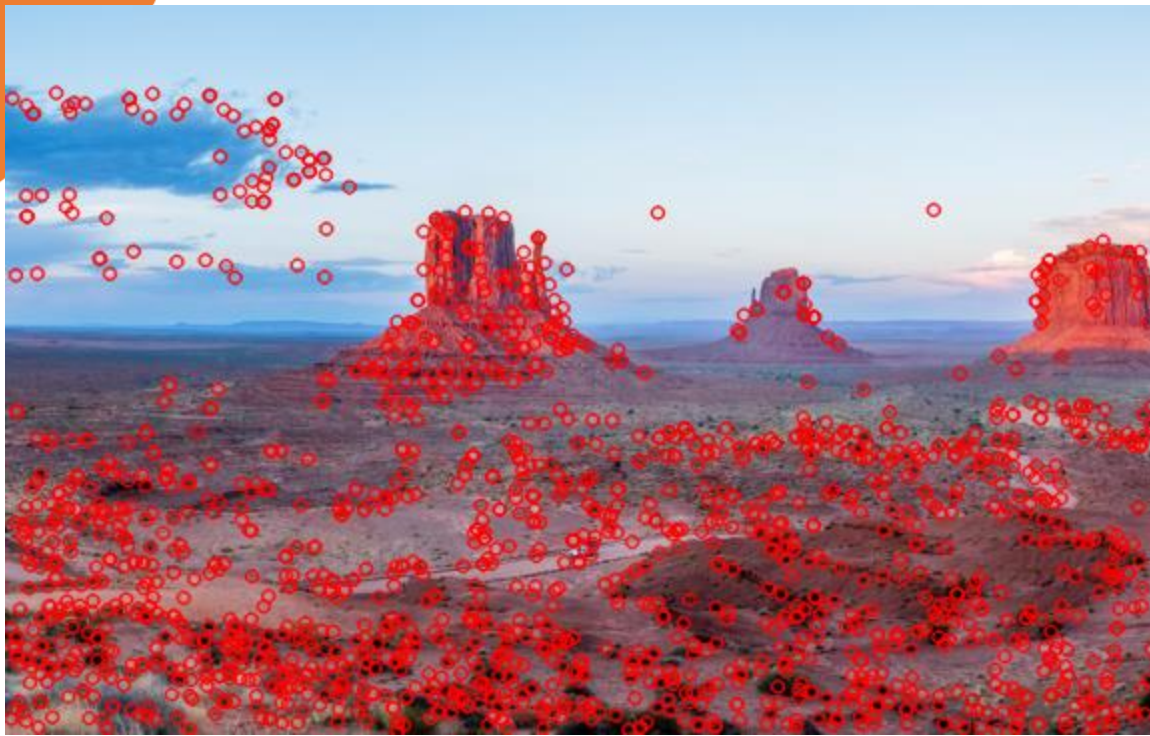
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

TABLE 2.3
Affine
transformations
based on
Eq. (2-45).

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{A} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine transformations

Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = y$	
Scaling/Reflection (For reflection, set one scaling factor to -1 and the other to 0)	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = c_x x$ $y' = c_y y$	
Rotation (about the origin)	$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x \cos \theta - y \sin \theta$ $y' = x \sin \theta + y \cos \theta$	
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + t_x$ $y' = y + t_y$	
Shear (vertical)	$\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + s_v y$ $y' = y$	
Shear (horizontal)	$\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = s_h x + y$	



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \phi_{1,1} & \phi_{1,2} & \tau_x \\ \phi_{2,1} & \phi_{2,2} & \tau_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine transformation

Affine transformation – rotation, translation, scale and shear

- Recall the planar generalized affine transformation in homogeneous coordinates

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \phi_{1,1} & \phi_{1,2} & \tau_x \\ \phi_{2,1} & \phi_{2,2} & \tau_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- We can find a solution for these linear equations

$$\begin{aligned} \phi_{1,1}x + \phi_{1,2}y + \tau_x &= x' \\ \phi_{2,1}x + \phi_{2,2}y + \tau_y &= y' \end{aligned}$$

Affine transformation

Translate from coordinates, $[x, y]$, in one image to another, $[x', y']$

- Solve **least squares problem** for **parameter vector**, Φ

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_n & y_n & 1 \end{bmatrix} \begin{bmatrix} \phi_{1,1} \\ \phi_{1,2} \\ \tau_x \\ \phi_{2,1} \\ \phi_{2,2} \\ \tau_y \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix}$$

$$A\Phi = y$$

- Also a **linear model**!

Projective transformation

Projective transform or homography - general unrestricted transformation

- Starting with linear system of equations

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \phi_{1,1} & \phi_{1,2} & \phi_{1,3} \\ \phi_{2,1} & \phi_{2,2} & \phi_{2,3} \\ \phi_{3,1} & \phi_{3,2} & \phi_{3,3} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Solution is **nonlinear** in Cartesian coordinates:

$$x' = \frac{\phi_{1,1}x + \phi_{1,2}y + \phi_{1,3}}{\phi_{3,1}x + \phi_{3,2}y + \phi_{3,3}}$$

$$y' = \frac{\phi_{2,1}x + \phi_{2,2}y + \phi_{2,3}}{\phi_{3,1}x + \phi_{3,2}y + \phi_{3,3}}$$

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} = \begin{bmatrix} \frac{x}{w} \\ \frac{y}{w} \\ 1 \end{bmatrix}$$

Projective transformation

Projective transform or homography - general unrestricted transformation

- Start with solution for $[x', y']$ given $[x, y]$:

$$x' = \frac{\phi_{1,1}x + \phi_{1,2}y + \phi_{1,3}}{\phi_{3,1}x + \phi_{3,2}y + \phi_{3,3}}$$

$$y' = \frac{\phi_{2,1}x + \phi_{2,2}y + \phi_{2,3}}{\phi_{3,1}x + \phi_{3,2}y + \phi_{3,3}}$$

- Rearrange terms to get linear form:

$$\phi_{1,1}x + \phi_{1,2}y + \phi_{1,3} - \phi_{3,1}xx' - \phi_{3,2}yx' - \phi_{3,3}x' = 0$$

$$\phi_{2,1}x + \phi_{2,2}y + \phi_{2,3} - \phi_{3,1}xy' - \phi_{3,2}yy' - \phi_{3,3}y' = 0$$

Projective transformation

Transform from coordinates, $[x, y]$, in one image to another, $[x', y']$

- Solve for **parameter vector**, Φ

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1x'_1 & -y_1x'_1 & -x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -x_1y'_1 & -y_1y'_1 & -y'_1 \\ & & & \vdots & \vdots & \vdots & & & \\ & & & \vdots & \vdots & \vdots & & & \\ x_n & y_n & 1 & 0 & 0 & 0 & -x_nx'_n & -y_nx'_n & -x'_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -x_ny'_n & -y_ny'_n & -y'_n \end{bmatrix} \begin{bmatrix} \phi_{1,1} \\ \phi_{1,2} \\ \phi_{1,3} \\ \phi_{2,1} \\ \phi_{2,2} \\ \phi_{2,3} \\ \phi_{3,1} \\ \phi_{3,2} \\ \phi_{3,3} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$A\Phi = 0$$

- A homogenous set of equations!

Projective transformation

Transform from coordinates, $[x, y]$, in one image to another, $[x', y']$

- Solve for **parameter vector**, Φ

$$A\Phi = 0$$

- Use the **direct linear transform (DLT)** algorithm to solve

Projective transformation

Direct Linear Transform (DLT) algorithm

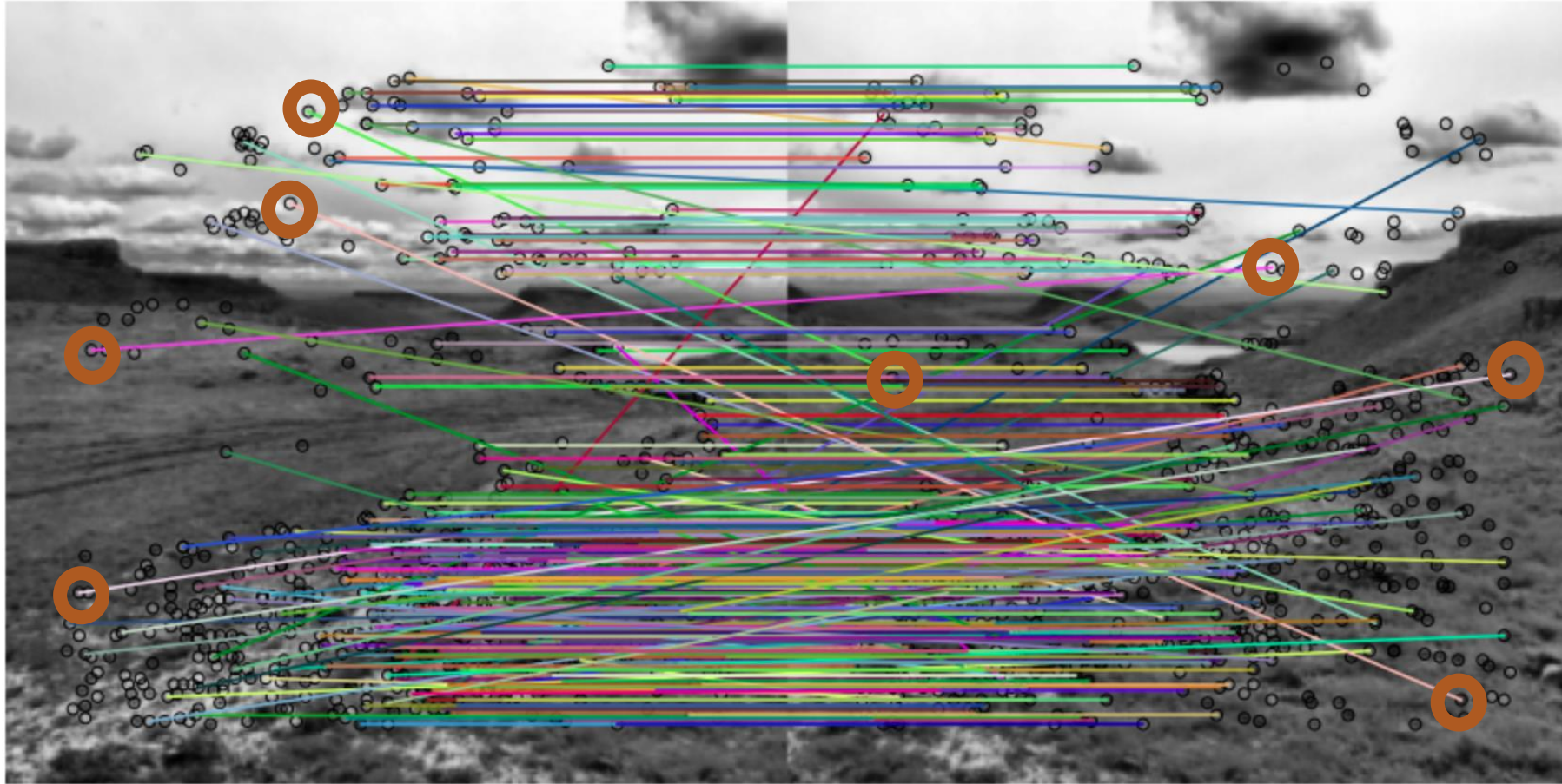
1. For each of n matched key-points create matrix A
 - A $2n \times 9$ matrix
2. Perform singular value decomposition (SVD) of matrix A
 - Find smallest non-zero singular value
 - Singular vector associated with smallest singular value is the solution, Φ
3. Construct homograph, H , from parameters Φ



Outliers and RANSAC

Solving for the transformation parameters

$[x, y], [x', y']$, pairs found by using matched descriptors

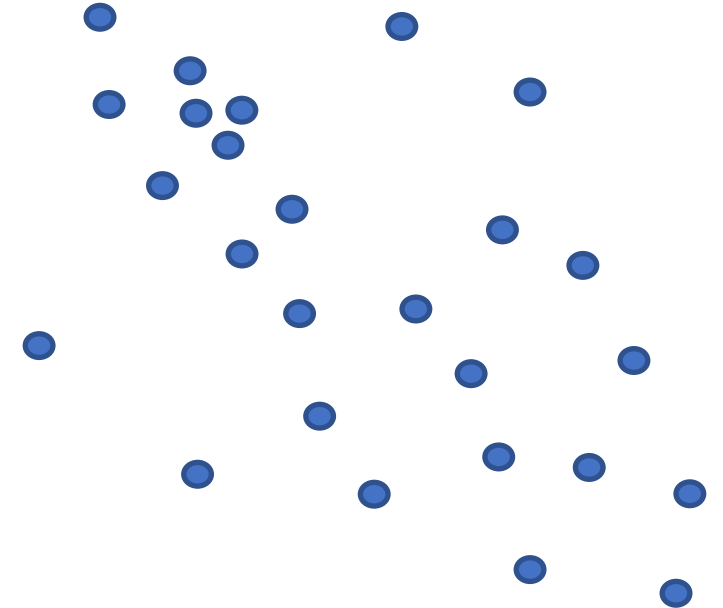


Notice the obvious matching errors – These are **outliers**!

Solving for the transformation parameters

RANdom SAMpling Consensus ([RANSAC](#)) algorithm example

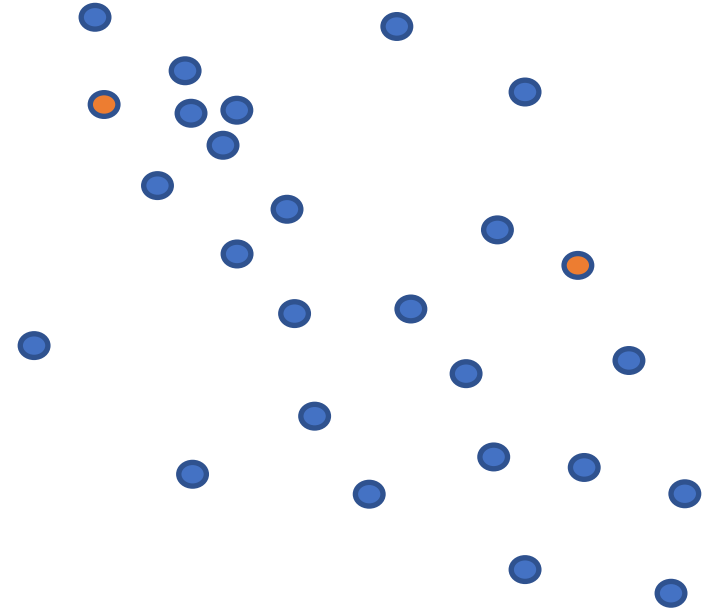
1. $s = 3$
2. Let DoF = 2



Solving for the transformation parameters

Random SAMpling Consensus ([RANSAC](#)) algorithm example

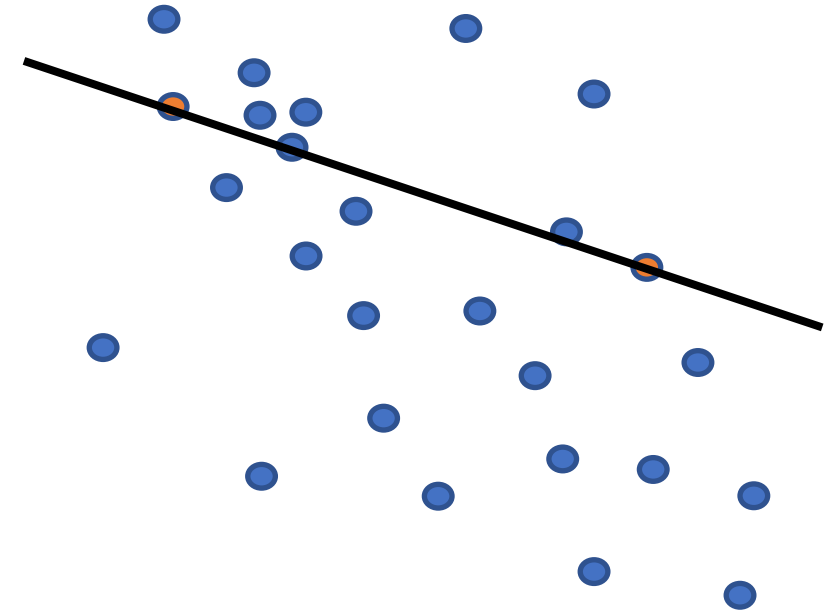
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random



Solving for the transformation parameters

RANdom SAMpling Consensus ([RANSAC](#)) algorithm example

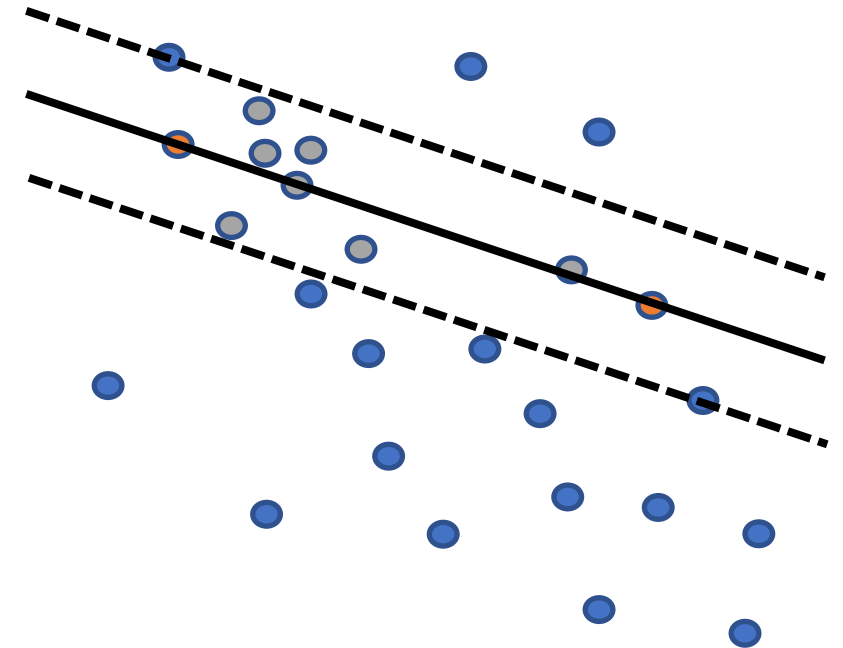
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random
4. Fit line



Solving for the transformation parameters

RANdom SAMpling Consensus ([RANSAC](#)) algorithm example

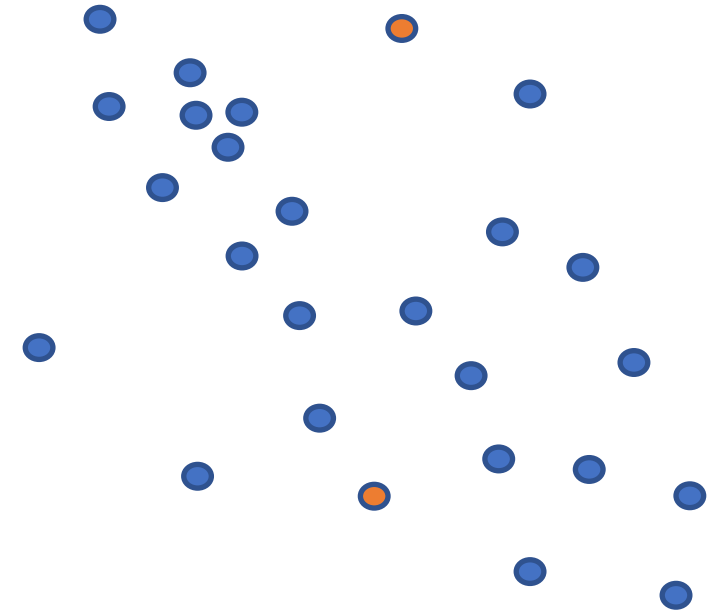
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random
4. Fit line
5. Identify inliers = 7



Solving for the transformation parameters

Random SAMpling Consensus (RANSAC) algorithm example

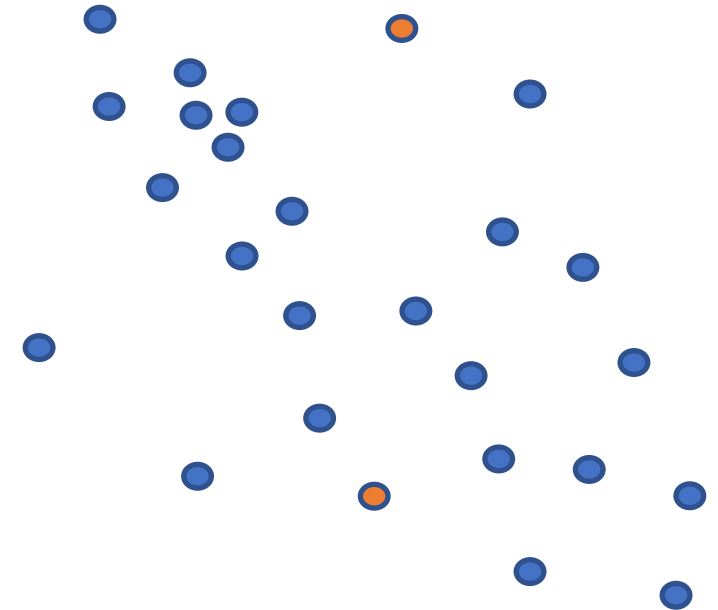
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random



Solving for the transformation parameters

Random SAMpling Consensus ([RANSAC](#)) algorithm example

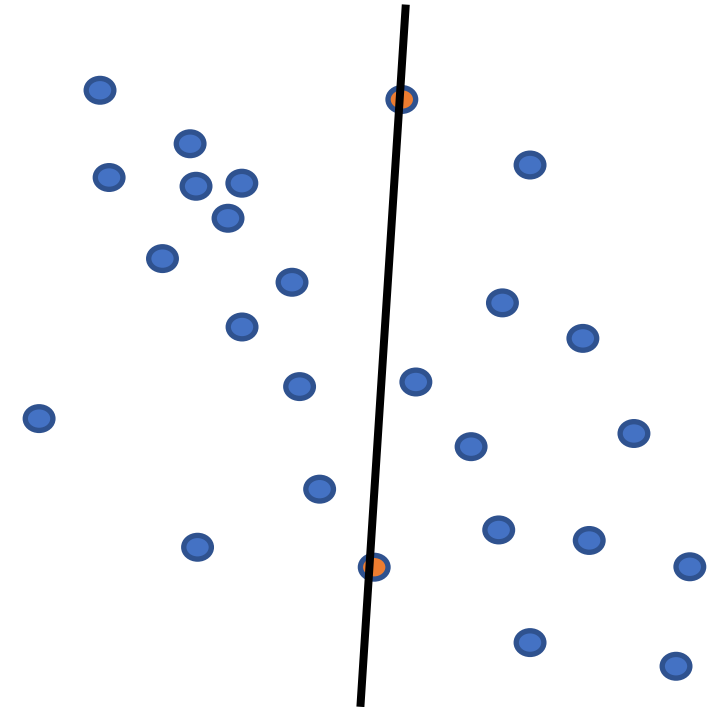
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random



Solving for the transformation parameters

RANdom SAMpling Consensus ([RANSAC](#)) algorithm example

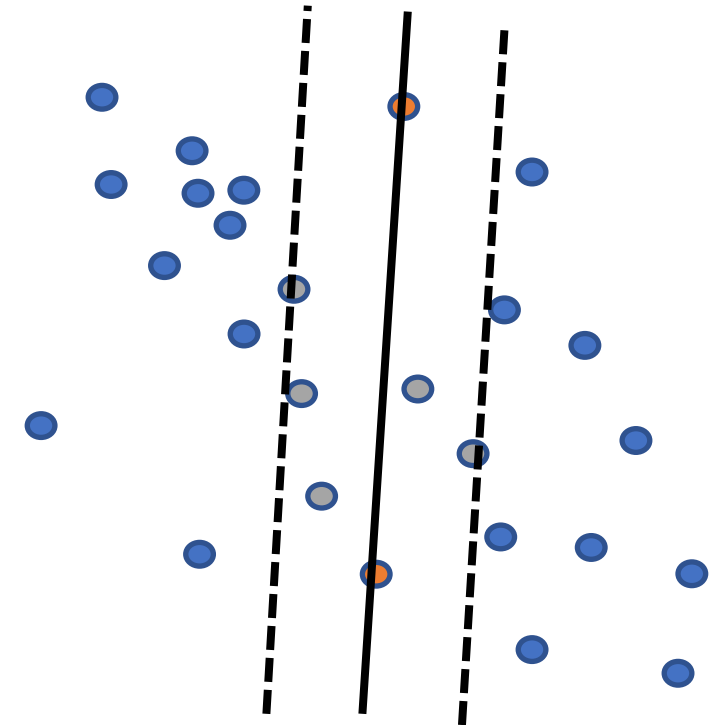
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random
4. Fit line



Solving for the transformation parameters

Random SAMpling Consensus (RANSAC) algorithm example

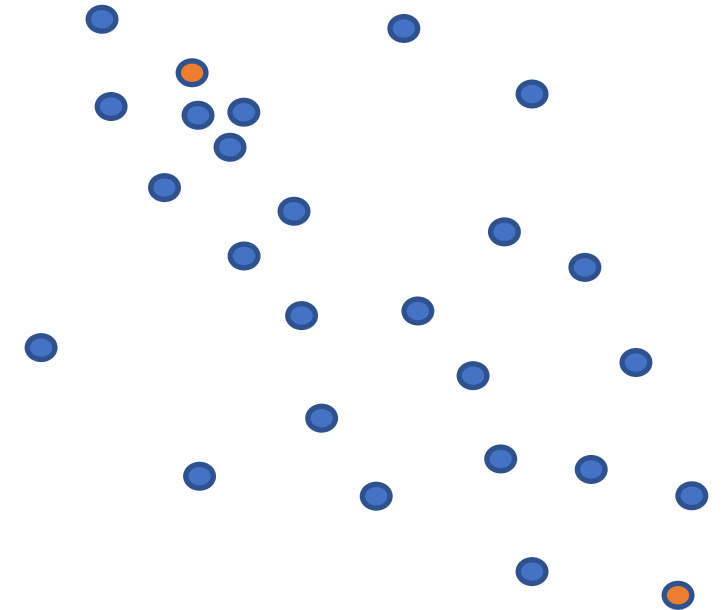
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random
4. Fit line
5. Identify inliers = 5



Solving for the transformation parameters

Random SAMpling Consensus ([RANSAC](#)) algorithm example

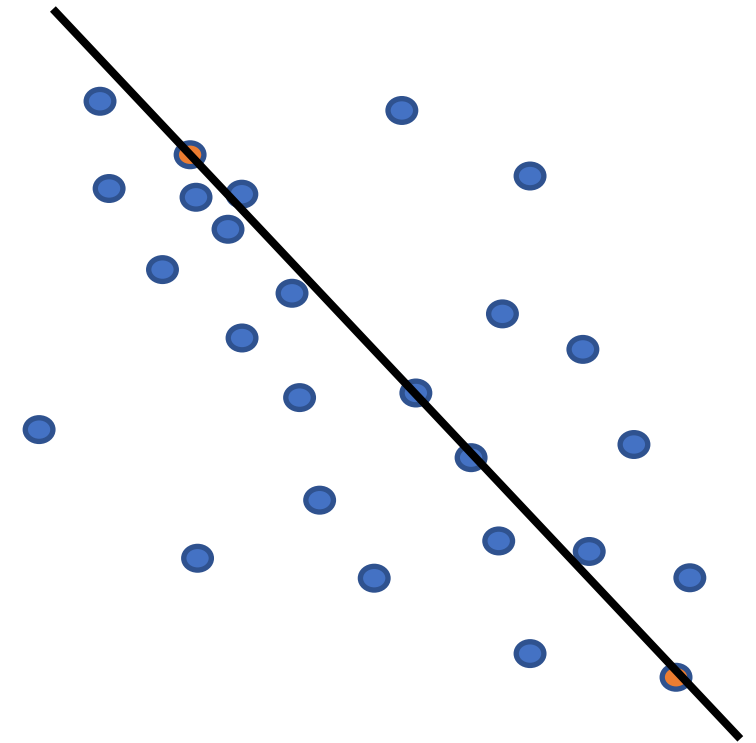
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random



Solving for the transformation parameters

RANdom SAMpling Consensus ([RANSAC](#)) algorithm example

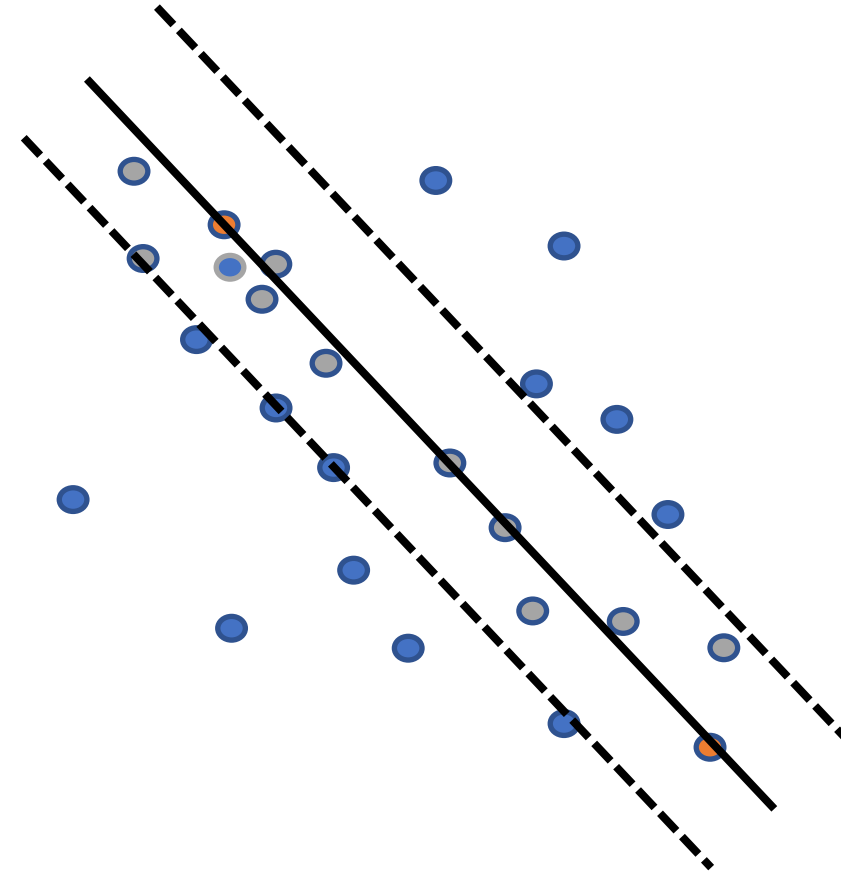
1. $s = 3$
2. Let DoF = 2
3. Select 2 points at random
4. Fit line



Solving for the transformation parameters

RANdom SAMpling Consensus ([RANSAC](#)) algorithm example

1. $S = 3$
2. Let DoF = 2
3. Select 2 points at random
4. Fit line
5. Identify inliers = 11



Solving for the transformation parameters

RAndom SAmpling Consensus ([RANSAC](#)) algorithm

- RANSAC algorithm uses a resampling algorithm to find a fit with largest number of inliers

N = number of points sampled = degrees of freedom

ϵ = tolerance

s = number of independent samples

For range(s):

 Select N pairs of matches

 Compute homography, $\mathcal{H}$

 Compute inliers: $\|x'_i - \mathcal{H}x_i\| < \epsilon$

 Save $\mathcal{H}$ and inliers

Select solution with largest number of inliers

Updated $\mathcal{H}$ using all inliers

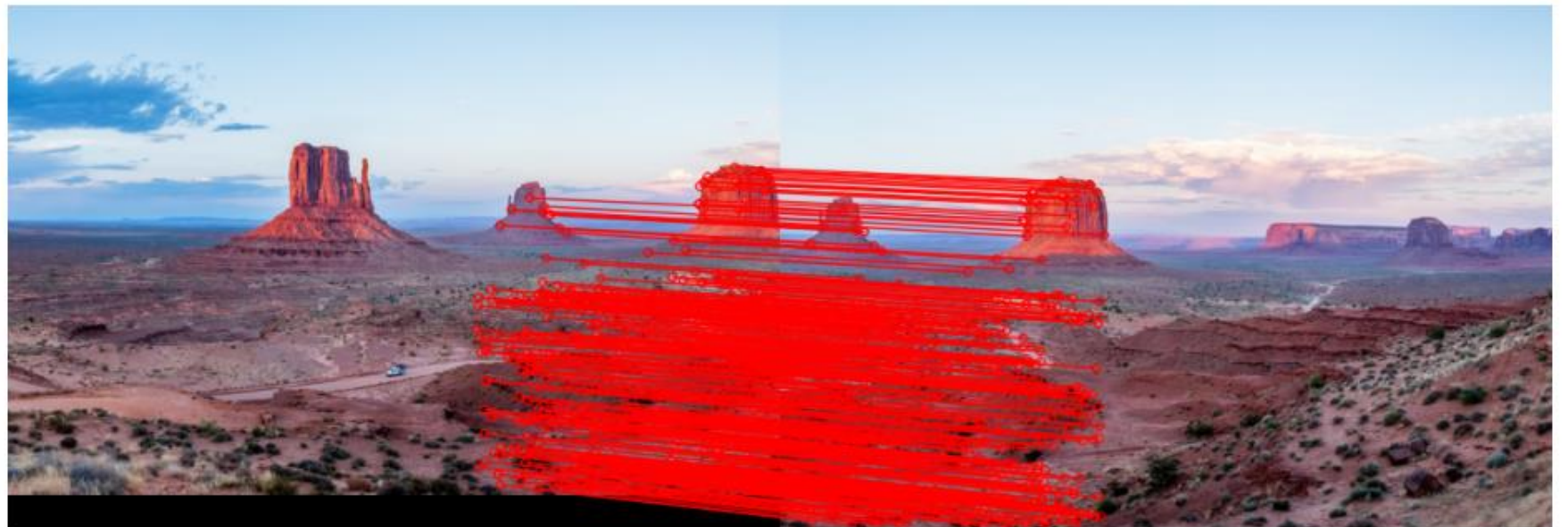
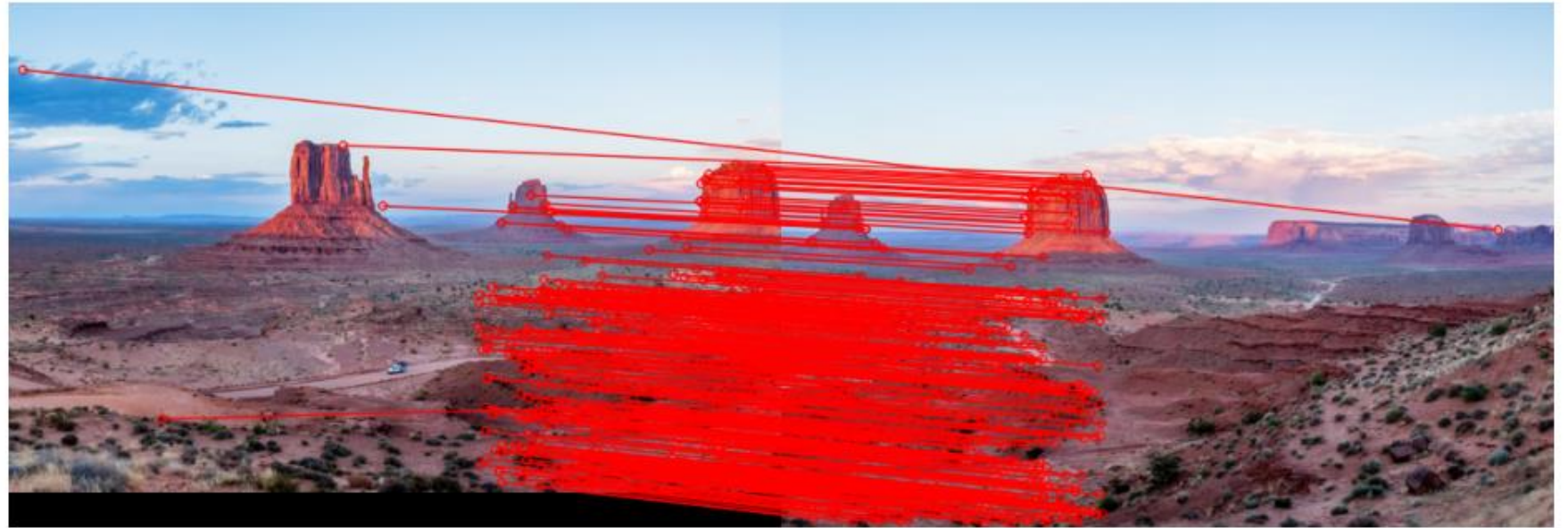




Image Warping

Image Warping

Use homography, $\mathcal{H}$, to warp one image to coordinate system to another

- Can use forward mapping of pixels between coordinate systems, $x \Rightarrow x'$

$$g(x') = \mathcal{H}(f(x))$$

- Can use inverse mapping of pixels between coordinate systems, $x' \Rightarrow x$

$$f(x) = \mathcal{H}^{-1}(g(x'))$$

- Which direction should you map?
- Depends on which image is the source and which is reference – Be careful!
- Usually apply the inverse transform

Image Warping

Use homography, $\mathcal{H}$, to warp one image to coordinate system to another

- When warping an image there is no guarantee that source pixels land within target grid boundaries – no constraint

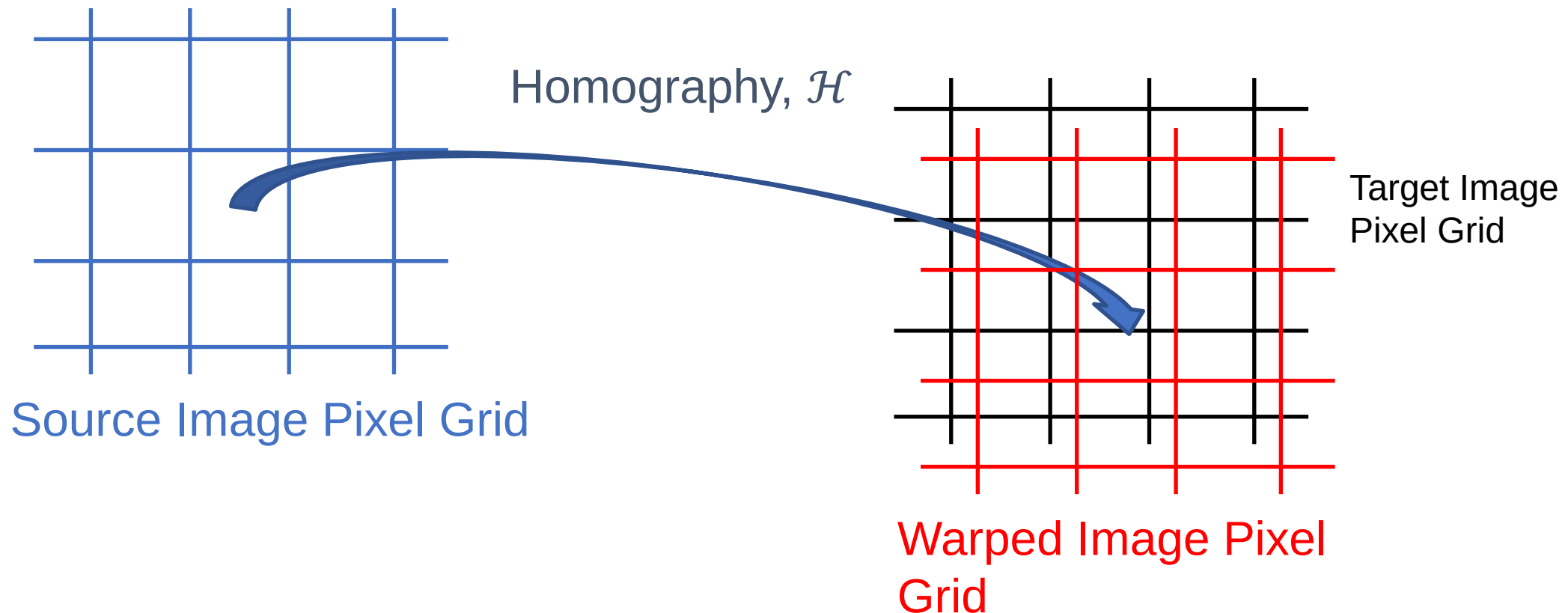


Image Warping

Use homography, $\mathcal{H}$, to warp one image to coordinate system to another

- When warping an image there is no guarantee that source pixels land within target grid boundaries - no constraint
- Values of warped pixel fall into several target pixels
- Warped pixel values influence several
- **Can interpolate!**
- Or apply other Compositing methods

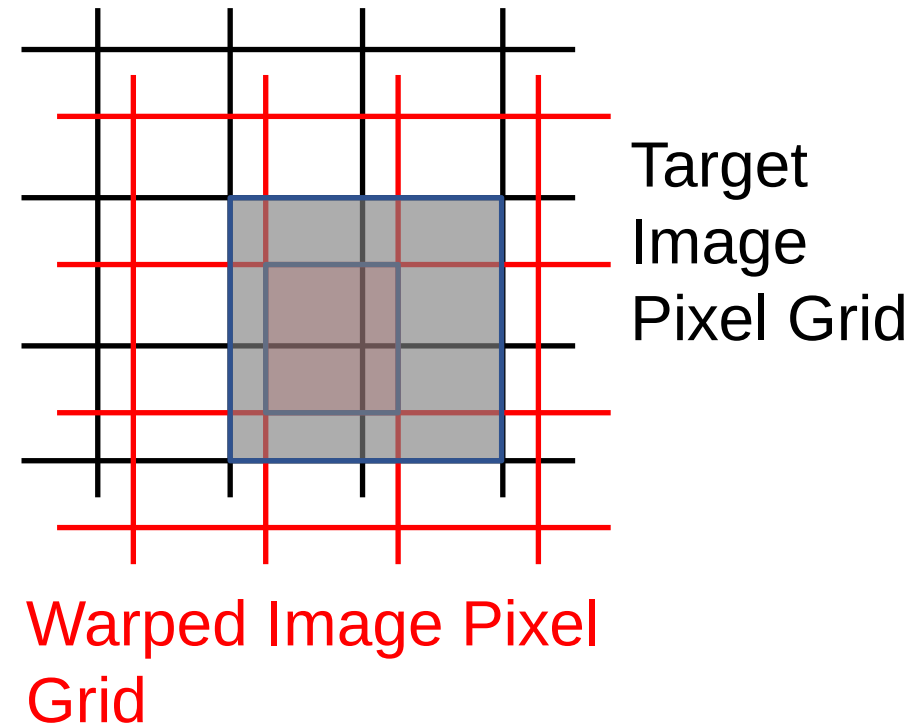


Image Warping

When warping an image there is no guarantee that source pixels land within target grid boundaries - no constraint

- Interpolate fill pixel values
- Scikit Image supports several of possible methods
 1. Nearest-neighbor
 2. Bi-linear (default)
 3. Bi-quadratic
 4. Bi-cubic
 5. Bi-quartic
 6. Bi-quintic

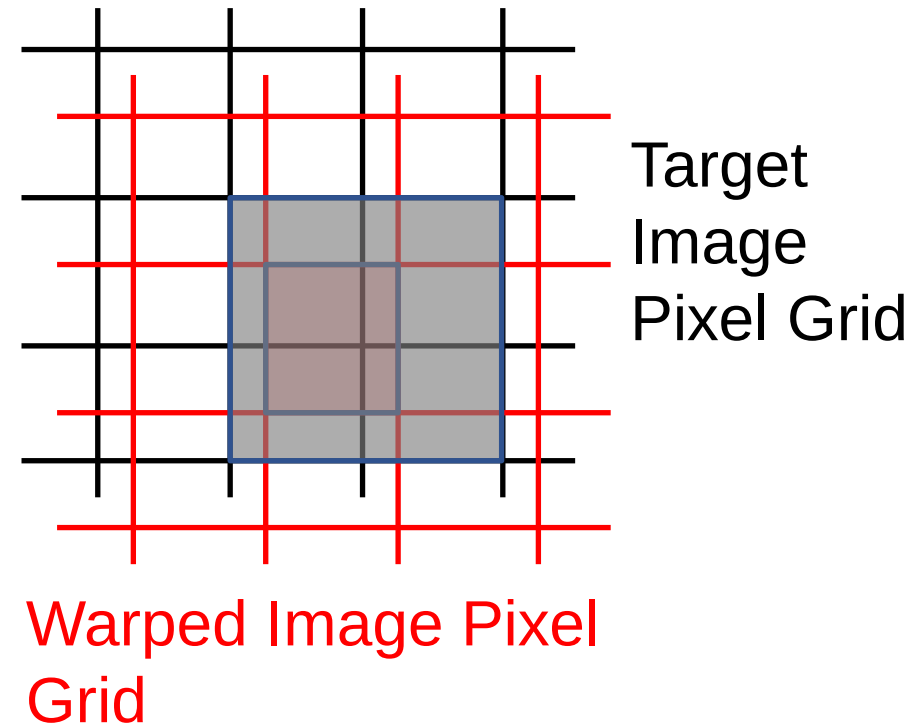
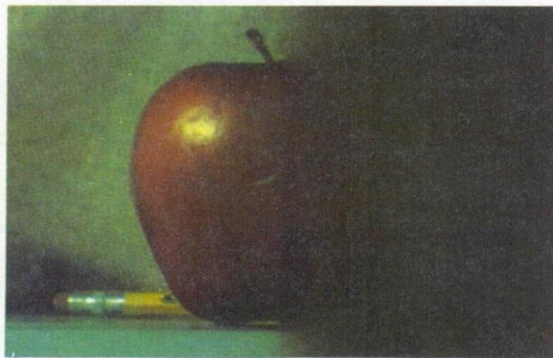
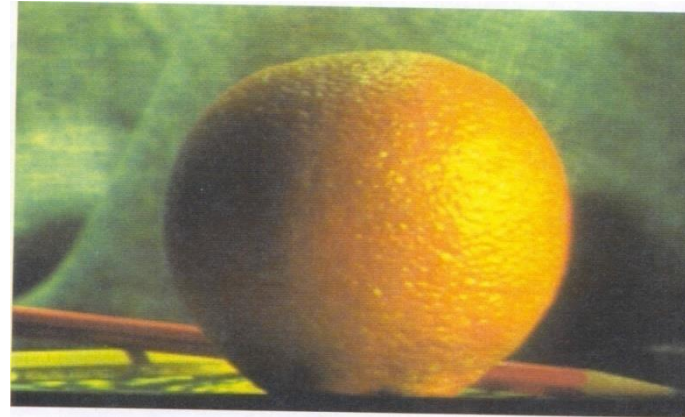
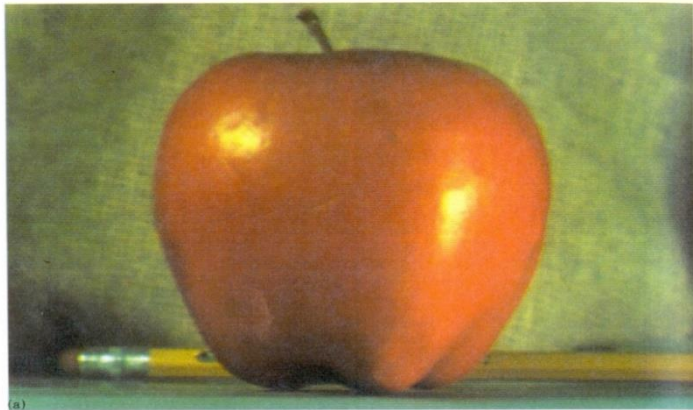




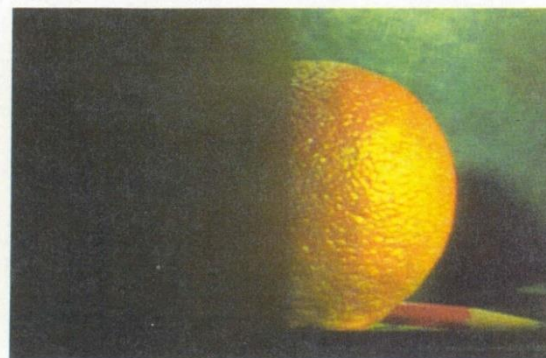
Image Blending

Image Blending

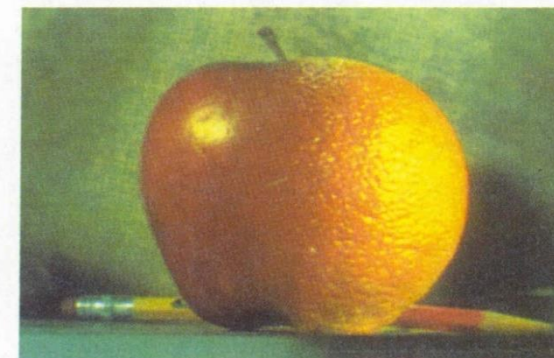
Image blending with scale pyramid is used in computer graphics



(d)



(h)

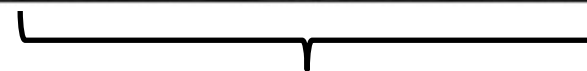


(l)

Burt, P. J. and Adelson, E. H., [A multiresolution spline with applications to image mosaics](#), ACM Transactions on Graphics, 42(4), October 1983, 217-236.

Image Blending

How can we make the transitions in the mosaic less noticeable?



Region of overlap

Image Blending

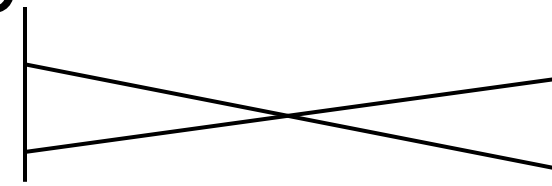
How can we make the transitions in the mosaic less noticeable?

- Blend one image into another over the region of overlap
 - Resulting pixel values are weighted sum of overlapping image pixel values
 - Blend different scales with pyramid
 - Blend R,G,B channels
 - Different blending function, linear, Gaussian, etc.
- Adjust illumination
 - Equalize illumination between images in mosaic
 - Can still be problems, e.g. shadows

Image Blending

Use interpolation to find weights

1. Sum of interpolation weights must add to 1.0
2. Linearly weight pixel values over range of overlap
3. Sum weighted pixel values to form blended



Region of

Image Blending

How can we make the transitions in the mosaic less noticeable?

- Compute Gaussian Laplacian pyramids of images
- Pyramid for each color channel
- Compute blending weights for each scale
- Blend Laplacians using weights
- Construct mosaic using weighted scales from pyramids
- [Example of image stitching with blending using Scikit Image in GitHub](#)

Summary

Key points for this lesson

- The goal of image stitching is to create a mosaic image from multiple original images
- Overlapping portions of images are matched by key point descriptors
- Transformations are computed by solution of linear of equations
 - For projective transform use DLT algorithm
- Use RANSAC to find solution with large number of outliers
- Images transformed and pixels interpolated
- Image blending to improve result